

Running OH AI Agent on Your Computer

This guide walks you through running the **OH AI Agent** on your own machine. You do not need prior Python experience, and you do **not** need Git — the project is supplied as a **zip file** that you extract to a folder on your computer. The zip includes a ready-to-use **.env** file with **OpenRouter** already configured. Steps are written for **Windows** first; macOS and Linux notes appear where commands differ.

The application has two parts that run at the same time:

Part	What it is	URL when running
Backend	Python API (data, AI, knowledge base)	http://localhost:8000
Frontend	Web dashboard in the browser	http://localhost:3000

You will open **two terminal windows** (or tabs): one for the backend, one for the frontend.

What you need before you start

- 1. **The project zip file** you received (for example **oh-ai-agent.zip**). It includes a pre-configured **.env** file with an **OpenRouter** API key — no separate sign-up is required.
- 2. **A computer** with administrator rights (to install software).
- 3. **Internet access** — to download uv and Node.js during setup, and for the app to call OpenRouter when you generate workflows.
- 4. **About 2 GB free disk space** for the extracted project, tools, dependencies, and the local knowledge index.

Step 1: Extract the project zip file

Pick a folder where you will keep the project (for example **Documents** on Windows). Extract the zip there.

Windows (File Explorer — recommended for beginners)

- 1. Move **oh-ai-agent.zip** to your chosen folder (for example **Documents**).
- 2. Right-click the zip file → **Extract All...**
- 3. Accept the destination folder and click **Extract**.
- 4. Open the extracted folder. You should see files such as **README.md**, **.env**, **pyproject.toml**, and a **frontend** folder.

Windows (PowerShell)

If the zip is in Downloads:

```
Expand-Archive -Path "$HOME\Downloads\oh-ai-agent.zip" -DestinationPath "$HOME\Documents" -Force
cd "$HOME\Documents\oh-ai-agent"
```

Change the zip path and folder name if yours differ.

macOS

Double-click the zip in Finder, or in Terminal:

```
unzip ~/Downloads/oh-ai-agent.zip -d ~/Documents
cd ~/Documents/oh-ai-agent
```

Linux

```
unzip ~/Downloads/oh-ai-agent.zip -d ~/Documents
cd ~/Documents/oh-ai-agent
```

Important: All later commands assume your **project root** is the folder that contains `pyproject.toml` and the `frontend` directory. If the zip created an extra nested folder (for example `oh-ai-agent/oh-ai-agent/`), open the inner one that contains those files.

Open a terminal in that folder before continuing:

Windows: In File Explorer, click the address bar, type `powershell`, press Enter.

macOS: Right-click the folder in Finder → **New Terminal at Folder** (if available), or `cd` to the path manually.

Step 2: Install uv (Python and backend dependencies)

This project uses `uv` to install Python and all backend libraries. You do **not** need to install Python separately first; `uv` can download the correct Python version for you.

Windows (PowerShell)

Run this in PowerShell (not Command Prompt unless you prefer it):

```
powershell -ExecutionPolicy ByPass -c "irm
https://astral.sh/uv/install.ps1 | iex"
```

Close and reopen PowerShell, then check:

```
uv --version
```

macOS / Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Restart your terminal, then:

```
uv --version
```

Step 3: Install Node.js (frontend)

The dashboard is built with **Node.js** and **npm** (Node's package manager).

1. Go to <https://nodejs.org/>.
2. Download the **LTS** (Long Term Support) installer — currently labelled something like “20.x LTS” or “22.x LTS”.
3. Run the installer. On Windows, leave “**Add to PATH**” enabled.
4. Restart your terminal.

Check both tools:

```
node --version  
npm --version
```

Each command should print a version (for example **v22.11.0** and **10.9.0**).

Step 4: Install backend dependencies

From the **oh-ai-agent** folder (project root):

```
uv sync
```

What this does (you do not need to memorise this):

- Creates a virtual environment (**.venv**) — an isolated Python environment for this project only.
- Installs Python **3.12** if it is not already available (see **.python-version** in the repo).
- Installs FastAPI, ChromaDB, LiteLLM, and other packages listed in **pyproject.toml**.

The first run can take several minutes. Wait until the command finishes without errors.

Step 5: Check the included **.env** file (no editing required)

The zip already contains a **.env** file in the project root. It configures the app to use **OpenRouter** with a working API key. **You do not need to copy **.env.example**, sign up for OpenRouter, or paste a key**

yourself.

Confirm the file is present after extracting:

Windows (PowerShell), from the project root:

```
Test-Path .env
```

This should print **True**.

Windows (File Explorer): **.env** sits next to **README.md**. If you do not see it, open the **View** tab and enable **Hidden items** (some tools hide dot-files).

The important settings are already set, for example:

```
OH_LLM_PROVIDER=openrouter  
OH_LLM_API_KEY=sk-or-v1-...  
OH_LLM_MODEL=openai/gpt-4o-mini
```

You can open **.env** in Notepad to verify it is not empty, but **do not change it** unless you are told to.

Keep the key private

The **.env** file contains a live API key paid for by the project provider.

- Do **not** upload the zip or **.env** to public websites, GitHub, or shared drives without permission.
- Do **not** paste the key into emails, chat messages, or screenshots.
- If you think the key was exposed, contact whoever supplied the zip so it can be rotated.

Optional: change the AI model

Advanced users only: edit **OH_LLM_MODEL** in **.env** (see comments in **.env.example** for other OpenRouter model ids). Restart the backend after any change.

Step 6: Install frontend dependencies

Open a terminal in the project root, then:

Windows (PowerShell):

```
cd frontend  
npm install  
cd ..
```

macOS / Linux: same commands.

`npm install` downloads React, Next.js, and UI libraries into `frontend/node_modules`. This may take a few minutes.

Step 7: Start the backend (Terminal 1)

Stay in (or return to) the **project root** `oh-ai-agent` — **not** inside `frontend`.

```
uv run uvicorn oh_agent.main:app --reload --host 0.0.0.0 --port 8000
```

Leave this window **open and running**. You should see log lines ending with something like “Uvicorn running on http://0.0.0.0:8000”.

First startup: The app may ingest documents from the `knowledge_base/` folder into a local database (`.chroma_db/`). That is normal.

Check the backend

Open a browser and visit:

- Health: <http://localhost:8000/health>
- API docs: <http://localhost:8000/docs>
- Config info (no secret keys shown): <http://localhost:8000/info>

If `/health` returns JSON with `"status": "healthy"` (or similar), the backend is working.

Step 8: Start the frontend (Terminal 2)

Open a **second** terminal window or tab.

Windows: `Win` → type `PowerShell` → Enter again.

From the project root, go to the frontend folder and start the dev server:

```
cd frontend
npm run dev
```

If Terminal 2 is not already in the project root, use the full path to your extracted folder first (for example `cd $HOME\Documents\oh-ai-agent`), then `cd frontend`.

Leave this window open. When ready, you should see a line mentioning `http://localhost:3000`.

Open the dashboard

In your browser, go to <http://localhost:3000>.

The UI talks to the backend at `http://localhost:8000` by default. If you change the backend port, set `NEXT_PUBLIC_API_URL` before `npm run dev` (see [Changing the API URL](#)).

Step 9: Confirm everything works

Use this checklist:

Check	How
Backend running	http://localhost:8000/health loads in the browser
Frontend running	http://localhost:3000 shows the dashboard
LLM configured	http://localhost:8000/info shows <code>llm_provider: openrouter</code> and a model name
Generate a workflow	In the UI, open Workflow Generator , fill the form, submit — should succeed with the included OpenRouter key

If workflow generation fails with **402** or **502**, see [Troubleshooting](#) — usually a missing `.env`, no internet, or exhausted API quota.

Daily workflow (after first setup)

You only repeat installation when you receive an **updated zip** or when setup instructions tell you to refresh dependencies. Extract the new zip (or replace the project files), then run `uv sync` and `npm install` in `frontend` again if needed. If the update zip does not include `.env`, keep your existing `.env` file or ask your provider for a fresh copy.

Each time you develop or demo:

1. **Terminal 1** (project root):
`uv run uvicorn oh_agent.main:app --reload --host 0.0.0.0 --port 8000`
2. **Terminal 2** (`frontend` folder):
`npm run dev`
3. Browser: <http://localhost:3000>
4. To stop: focus each terminal and press `Ctrl+C`.

Windows tips

Which program to use for commands

Use **PowerShell** or **Windows Terminal** with PowerShell. Commands in this guide use the same syntax in both.

Paths with spaces

If your project folder path has spaces, wrap it in quotes:

```
cd "C:\Users\YourName\My Projects\oh-ai-agent"
```

Firewall

The first time you run the servers, Windows may ask to allow Python or Node on private networks. Choose **Allow** for local development on your home or office network.

`.env` file missing after extract

If `Test-Path .env` prints `False`, the zip may be incomplete. Contact whoever supplied the zip and ask for a copy that includes `.env`.

Port already in use

If you see "address already in use" for port **8000** or **3000**:

1. Stop any old terminal still running the app (`Ctrl+C`).
2. Or find and stop the process:
 - PowerShell (port 8000):
`Get-NetTCPConnection -LocalPort 8000 -ErrorAction SilentlyContinue`
 - Then close the app using that port, or restart the computer if unsure.

Changing the API URL

If the backend is not on port 8000, tell the frontend before starting it.

Windows (PowerShell):

```
cd frontend
$env:NEXT_PUBLIC_API_URL="http://localhost:8000"
npm run dev
```

macOS / Linux:

```
cd frontend
export NEXT_PUBLIC_API_URL=http://localhost:8000
npm run dev
```

Adding documents to the knowledge base

1. Put `.txt`, `.md`, or `.docx` files in the `knowledge_base/` folder at the project root.
2. Restart the backend, **or** use the **Knowledge Base** page in the UI to upload or re-ingest.

Embeddings are stored under `.chroma_db/` on your machine (no separate database server).

Troubleshooting

uv: command not found or npm: command not found

- Install the tool (Steps 2 and 3).
- **Close and reopen** the terminal so PATH updates apply.
- On Windows, confirm Node installer added Node to PATH.

uv sync fails with a Python or compiler error

- Update uv: reinstall from <https://docs.astral.sh/uv/getting-started/installation/>.
- Ensure you are in the `oh-ai-agent` root (folder containing `pyproject.toml`).
- On Windows, if a package needs a C compiler, install [Visual Studio Build Tools](#) with "Desktop development with C++" — only if errors mention compiling native code.

Backend starts but workflow returns 402 / 502

- Confirm `.env` exists in the project root (`Test-Path .env → True`) and was not edited accidentally.
- Confirm the computer has internet access (OpenRouter is a cloud service).
- Visit <http://localhost:8000/info> and check `llm_provider` is `openrouter` and `llm_resolved_model` looks correct.
- If problems persist, contact whoever supplied the zip — the shared key may need quota or rotation.

Frontend shows errors connecting to API

- Confirm Terminal 1 backend is still running.
- Open <http://localhost:8000/health> in the browser.
- Check `NEXT_PUBLIC_API_URL` matches your backend URL.

Browser shows a blank page on port 3000

- Read errors in Terminal 2 (frontend).
- Run `npm install` again inside `frontend`.
- Try a hard refresh: `Ctrl+Shift+R` (Windows) or `Cmd+Shift+R` (Mac).

Permission denied on `logs/` or `.chroma_db/`

- Run the terminal as your normal user (not required to be Administrator).
- Ensure the project folder is writable (not read-only); avoid running from `Program Files`.

Optional: run tests (verify your install)

From the project root:

```
uv run pytest tests/ -v
```

Frontend lint (from `frontend`):


```
npm run lint
```

Quick reference

Task	Directory	Command
Install backend deps	oh-ai-agent	uv sync
Install frontend deps	oh-ai-agent/frontend	npm install
Start backend	oh-ai-agent	uv run uvicorn oh_agent.main:app --reload --host 0.0.0.0 --port 8000
Start frontend	oh-ai-agent/frontend	npm run dev
API documentation	Browser	http://localhost:8000/docs
Dashboard	Browser	http://localhost:3000

Getting help

- Project overview and API list: [README.md](#)
- Environment variable reference (no secrets): [.env.example](#)

If something in this guide does not match your screen, note your operating system version, the exact command you ran, and the full error message when asking for support.